LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection

***Abstract*—As the use of third-party intellectual property (IP) cores and the complexity of hardware design workflows continue to grow, Hardware Trojan detection at the Register Transfer Level (RTL) is becoming increasingly critical. Many existing hardware deep learning (DL) and transformer-based approaches to RTL security analysis have high computational resource demands and do not support reproducibility, particularly in academic settings with limited resources. In this work, we present an engineered prototype, lightweight and reproducible, for RTL Trojan analysis using parameter-efficient large language model (LLM) adaptation. Our proposed approach employs LoRA-based fine-tuning of the Qwen2.5-1.5B-Instruct model for Trojan detection and localization in RTL designs. A custom synthetic RTL Trojan dataset of clean and Trojan-only Verilog samples was developed using controlled trigger and payload insertion. To prevent shortcut learning, we utilized both lightweight structural obfuscation and prompt normalization methods at the preprocessing stage. Evaluation was performed on a randomly stratified test split of the dataset within the controlled synthetic RTL environment and included both classification and localization analyses using Trigger Localization Coverage (TLC) and Payload Localization Coverage (PLC). The entire implementation was executed on a Google Colab Tesla T4 GPU using low-resource LoRA adaptation. The framework achieved 98.89% primary metric accuracy, 98.88% F1-score, 98.89% TLC, and 98.89% PLC metrics while also maintaining a computationally-efficient training setup suitable for reproducible experimentation.**

***Index Terms*—Hardware Trojan Detection, RTL Security, LoRA, Large Language Models, Parameter-Efficient Fine-Tuning**

I. Introduction

Contemporary integrated circuit (IC) design has seen an increasing reliance on third-party intellectual property (IP) cores, hardware reuse, and outsourced semiconductor manufacturing, which introduces significant security risks. Hardware Trojans (HTs) are often the most insidious threat in this environment because there is a wide possibility that a HT will remain dormant during standard verification testing before activation under rare operation conditions. As hardware systems become more structurally advanced, early Trojan analysis at the Register Transfer Level (RTL) will be increasingly critical because of the ability to represent the structural design characteristics of the RTL model prior to the synthesis and physical implementation of the IC [1], [2].

Typical HT detection techniques, such as functional testing, formal verification, side-channel analysis, and handcrafted machine learning, can effectively identify HTs in smaller circuits and based on known HT patterns; however, they don't generalize well across structurally dissimilar and obfuscated RTL designs. Additionally, reliance on manually developed features and/or assumed circuit behaviors will further limit the scalability and robustness of RTL security analysis in practical scenarios [1], [2].

Large language models (LLMs) based on transformers have recently exhibited considerable strengths in interpreting structured sequential representations. Hardware description languages (HDLs), like Verilog, exhibit similar structural properties to traditional programming languages, so therefore transformer architectures may be able to learn contextual RTL relationships directly from the serialized hardware description [3]. This has led to increasing interest in using LLMs for tasks related to RTL comprehension, hardware security assessment, and Trojan localization [4], [5].

Normally, modifying LLMs for RTL security applications requires large amounts of computation and high-memory GPU infrastructure; however, many parameter-efficient approaches to adaptation, such as LoRA, reduce the overhead by updating a limited subset of trainable parameters while retaining most of the pretrained model parameters intact [6].

The presented analysis framework for compact RTL Trojan analysis is based on the systematic and efficient use of Qwen2.5-1.5B-Instruct model adaptation to RTL Trojans. Overall, the complete implementation has been executed using LoRA-based adaptation on a Google Colab NVIDIA Tesla T4 GPU.

This work has made several significant contributions, including:

1) Development of a compact RTL preprocessing and prompt-engineering workflow for working with LLM-based Trojan analysis.
2) Construction of a synthetic RTL Trojan dataset with controllable trigger and payload insertion.
3) Implementation of parameter-efficient LoRA-based adaptation to support RTL Trojan detection and localization.
4) Evaluation of classification and localization capabilities under hardware-constrained training environments.

II. Related Work

Various techniques have been used extensively to study hardware Trojans, including functional testing, formal verification, side-channel analysis, and structural circuit analysis. The typical techniques used in Trojan detection involve looking for abnormal switching behaviour, timing deviations, power variations, or suspiciously modified structures in the design of the hardware circuit. These methods have worked reasonably

well for relatively small circuits with known Trojan characteristics but they have become increasingly challenging to apply to larger and more complex RTL designs that are just being designed and tested.

Machine learning and Deep Learning have been investigated as potential ways to improve security analysis of hardware in relation to hardware Trojans. Existing research has explored the use of supervised classifiers, graph-based learning methods, and transformer-inspired architectures to learn about RTL representations and detecting malicious changes in hardware [7], [8]. However, both machine learning and deep learning methods will still require substantial amounts of training data, high-memory GPU-capable hardware, and computationally intensive model adaptation processes, which can limit reproducibility of results in resource-limited research environments.

Numerous code-oriented language modeling approaches have been developed for analyzing hardware security vulnerabilities. Since Hardware Description Languages (HDLs), such as Verilog, have many of the same structural characteristics as programming languages, transformer models are potentially able to learn contextual relationships in register transfer level (RTL) logic using instruction-style representations and prompt-based learning methodologies [3]. Previous work (e.g., PEARL) used large language models (LLMs) for adaptive and explainable hardware Trojan detection, while TrojanWhisper leveraged pre-trained LLMs for RTL security risk assessment by detecting and localizing Trojans [4], [5].

While extensive fine-tuning methods are computationally expensive, the research at hand utilizes efficient and well-established domain adaptation methods—specifically, low-rank adaptation (LoRA)—to perform efficient and reproducible RTL Trojan detection analysis [6].

III. Proposed Methodology

A. RTL Trojan Analysis Pipeline

The RTL Trojan analysis framework proposed in this paper consists of five stages: RTL preprocessing, synthetic Trojan insertion, prompt preparation, parameter-efficient adaptation, and localization-aware inference. This workflow allows structured RTL security analysis using transformer-based language models within constrained GPU environments commonly available in academic research settings.

Clean RTL samples were collected from the RTL-Coder repository and transformed into instruction-oriented formats compatible with transformer-based processing workflows [9]. Synthetic trigger and payload logic were inserted into structurally valid RTL modules while preserving functional RTL behavior. Trigger and payload nodes generated during insertion were tracked as metadata for localization evaluation.

The RTL preprocessing stage included serialization, formatting normalization, whitespace cleanup, and obfuscation-aware transformations intended to reduce dependence on explicit RTL naming patterns.

The Qwen2.5-1.5B-Instruct model was used as the backbone model and adapted using LoRA-based parameter-efficient fine-tuning methods [6]. Most backbone parameters remained frozen while lightweight adapter layers were updated during training. The complete workflow was executed using Google Colab with an NVIDIA Tesla T4 GPU.

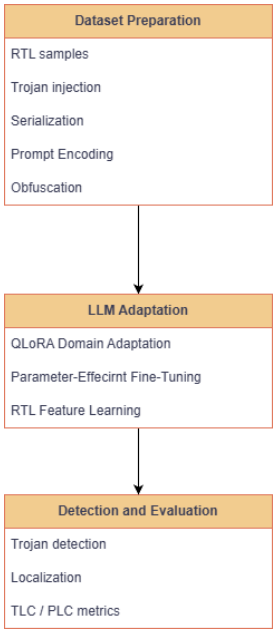


Fig. 1. Overall RTL Trojan analysis pipeline.

During inference, the adapted model performed Trojan classification together with trigger and payload localization analysis. Localization capability was evaluated using overlap-based TLC and PLC methodologies similar to those used in recent RTL Trojan analysis studies [4], [5].

B. Trojan Localization Strategy

The process used to localize Trojans followed a workflow that focused on identifying RTL regions associated with trigger conditions and malicious payload behavior from Trojan-inserted Verilog Modules (TIVMs). The localization was performed directly at the RTL structural level rather than at the post-synthesis gate level.

By controlling the synthetic Trojan insertion process, the authors were able to use metadata (i.e., `trigger_nodes` and `payload_nodes`) to conduct training and evaluation of the localization analysis. All RTL samples that had synthetically inserted Trojans had corresponding `trigger_nodes` and `payload_nodes` metadata that tracked the trigger activation logic and the RTL regions associated with the Trojan payload, respectively.

During the inference phase, the model utilized the serialized RTL prompts to predict the structurally suspicious RTL regions that were associated with the Trojan trigger and payload. The prediction of RTL regions generated during inference was compared to the tracked ground-truth metadata created during the synthetic Trojan insertion process using overlap-based analysis instead of exact structural matching. Similar

overlap-based localization analysis has also been explored in previous LLM-based RTL Trojan studies [4], [5].

Finally, Trigger Localization Coverage (TLC) and Payload Localization Coverage (PLC) metrics were computed using overlap-based coverage analysis between the predicted RTL regions and the corresponding ground-truth metadata.

C. Parameter-Efficient Adaptation Workflow

To minimize the resource use required to perform a full LLM adaptation for RTL analysis, both parameter-efficient fine-tuning and Low-Rank Adaptation (LoRA) PEFT methodologies were utilized.

The Qwen2.5-1.5B-Instruct model was selected as the base LLM because it contains a moderate parameter size and supports instruction-oriented learning workflows. During the adaptation phase, the majority of the backbone model parameters remained unchanged while lightweight adapter layers were introduced to support domain-specific RTL learning.The implementation utilized the HuggingFace Transformers framework together with the PEFT and bitsandbytes optimization libraries [10]–[12].

The complete adaptation workflow was executed within the Google Colab platform using an NVIDIA Tesla T4 GPU. By reducing the number of trainable parameters, the framework reduced memory allocation and computational overhead during adaptation.

IV. Experimental Setup

A. Dataset Engineering

The experimental dataset utilized in this study was created using clean samples of Verilog RTL available from the RTL-Coder repository [9]. The RTL-Coder dataset contains unique RTL implementations that have been formatted in an Instruction-Directed Representation and that can be used with transformer-based processing methods. The RTL modules in this repository represent a baseline for eventually generating synthetic Trojans and performing RTL Security Analysis.

Control of the RTL-level triggers and payloads was established such that valid RTL function was retained after Trojan insertion. All trigger and payload metadata was also recorded for each insertion to allow for Trigger and Payload Localization to be evaluated later. The final dataset contains a balanced number of clean and Trojan inserted RTL samples and closely match the RTL-oriented security analysis workflows that have been researched within other LLM-based studies on hardware security [4], [5].

To maintain a balanced evaluation, a randomized stratified split was performed across the training, validation and testing datasets. See Table I for an overview of the dataset distribution used for this experimental work.

TABLE I
Dataset Distribution

Dataset Split	Clean	Trojan	Total
Training	210	210	420
Validation	45	45	90
Testing	45	45	90
Total	300	300	600

B. Trojan Injection Strategy

Controlled synthetic insertion of trigger and payload logic into clean Verilog RTL modules was performed to produce Trojan-inserted RTL samples in a way that retained valid RTL functionality. The insertion process created standalone RTL areas (or regions) for the trigger activation and malicious payload behavior, which permitted structured RTL security analysis without altering the overall design format. This method's focus was on compact modifications at the RTL level, as opposed to the gate-level synthesis workflow used in industry, for the purpose of conducting controlled Trojan experimentation.

During generation, controlled trigger and payload logic were inserted as compact, standalone RTL regions. The trigger region represented the activation condition, while the payload region represented the malicious behavior. For each modified sample, the affected regions were recorded as `trigger_nodes` and `payload_nodes` metadata and retained separately from the serialized training prompts for localization evaluation. The deterministic procedure produced the 300 Trojan-inserted samples used alongside the 300 clean samples in the final balanced dataset.

A simplified RTL Trojan trigger insertion example is shown below:

```
reg htrojan_trig;
always @(*) begin
    htrojan_trig = (in == 4'h1);
end
```

For each modified RTL sample, `trigger_nodes` and `payload_nodes` metadata were tracked during the generation process of the Trojans. These metadata entries corresponded to RTL regions associated with the trigger activation condition and functionality of the associated payload. Since the Trojan contents were inserted in a deterministic manner, the same node-level annotations were available for the training and localization evaluation stages. Similar methodologies have also been investigated using metadata-based RTL Trojan analysis techniques in more recent LLM-based hardware security related studies [4], [5].

During the final prompt preparation stage, obfuscation-aware preprocessing was performed to limit dependency on the specific artifacts associated with the explicit Trojan insertion. The temporary HT-related markers that were used within the dataset generation process were removed prior to use for training in order to limit shortcut learning behavior to some degree. Finally, the serialized RTL prompt files retained their RTL structural relationships while providing no exposure to explicit Trojan annotations.

C. Obfuscation and Prompt Preparation

The RTL modules written in Verilog were modified to use a serialized text format that allowed processing via transformer-based workflows. The RTL examples were converted to an instruction/response format with normalization for formatting, whitespace removal, and common serialization conventions while keeping the structural relationship of the RTL intact. Similar preprocessing workflows associated with RTL models have also been utilized in various LLM hardware security investigations [4], [5].

To minimize the amount of information leakage during prompt preparation, the final prompt creation process included several sanitization procedures. To prevent shortcut learning, `HT_START` and `HT_END` indicators together with Trojan-specific comments were removed as part of the final training process while retaining the structural operation of the RTL code.

A simplified sanitization example is shown below:

```
// Before Sanitization
// HT_START
// Trojan Type: Payload_Override

// After Sanitization
module adder(...);
```

During evaluation of the adapted model, inference was completed using only the prompt information for generating RTL input with no disclosure of ground-truth response annotations to the model. Classification and localization targets were also hidden during the testing phase to encourage structural reasoning of the RTL code instead of using explicit annotation artifacts during Trojan investigation.

D. Training Environment and Configuration

The experimental procedure for training and evaluating the model incorporated a limited-resource architecture designed to allow for reproducible scientific experimentation on an academic level. The adaptation experiments were performed within a Google Colab environment utilizing an NVIDIA Tesla T4 GPU, allowing the complete RTL Trojan analysis workflow to operate without requiring dedicated high-memory server infrastructure. The low-resource setup also enabled practical experimentation using consumer-accessible GPU hardware commonly available in academic environments.

The Qwen2.5-1.5B-Instruct model was used for parameter-efficient adaptation together with a LoRA-based PEFT workflow [6], [11]. During training, the majority of the backbone model weights remained frozen while lightweight trainable adapter layers were introduced for domain-specific RTL adaptation. Initial exploratory experiments investigated QLoRA-style low-bit adaptation supported by bitsandbytes optimization utilities [12], [13]. However, the final randomized retraining workflow utilized FP16 LoRA adaptation in order to improve runtime stability and maintain reproducible training behavior. The overall implementation was developed using the HuggingFace Transformers framework together with PyTorch-based training utilities [10], [14].

TABLE II
TRAINING ENVIRONMENT AND CONFIGURATION

Component	Configuration
Base Model	Qwen2.5-1.5B-Instruct
Fine-Tuning Method	LoRA-based PEFT
Precision Type	FP16
LoRA Rank	8
Learning Rate	2e-4
Batch Size	1
Gradient Accumulation	8
Training Epochs	3
Trainable Parameters	2.17M
Trainable Percentage	0.141%
GPU Hardware	NVIDIA Tesla T4
Platform	Google Colab
Runtime	$\sim$6 minutes

The reduced number of trainable parameters enabled lightweight fine-tuning with lower computational overhead and short runtime execution, supporting practical reproducibility for low-resource experimentation. The complete workflow remained feasible within a single-session Google Colab environment without requiring distributed multi-GPU infrastructure or large-scale computing resources.

E. Evaluation Metrics

The ability of the adapted model to distinguish clean RTL samples from Trojan-inserted RTL samples during inference was evaluated using standard binary classification metrics including accuracy, precision, recall, and F1-score. Accuracy provided an overall indication of classification correctness, whereas precision and recall were used to assess false-positive and false-negative behavior associated with Trojan detection, respectively. The F1-score was included to provide a balanced evaluation of classification performance for RTL Trojan analysis.

In addition to binary classification metrics, localization performance was evaluated using Trigger Localization Coverage (TLC) and Payload Localization Coverage (PLC) methodologies [4], [5]. Both metrics employed overlap-based comparison between predicted RTL node regions and tracked ground-truth Trojan metadata generated during synthetic Trojan insertion. TLC was used to quantify overlap coverage between predicted and ground-truth trigger-related RTL nodes, while PLC quantified overlap coverage between predicted and ground-truth payload-related RTL nodes.

$$TLC = \frac{|T_p \cap T_g|}{|T_g|} \tag{1}$$

$$PLC = \frac{|P_p \cap P_g|}{|P_g|} \tag{2}$$

where T_p and P_p represent predicted trigger and payload nodes, while T_g and P_g represent the corresponding ground-truth trigger and payload nodes.

TABLE III
DETECTION PERFORMANCE METRICS

Metric	Value
Accuracy	98.89%
Precision	100.00%
Recall	97.78%
F1-Score	98.88%

TABLE IV
CONFUSION MATRIX

Actual Class	Predicted Clean	Predicted Trojan
Clean	45	0
Trojan	1	44

During evaluation, inference was performed using prompt-only RTL inputs while response annotations remained hidden from the adapted model. This leakage-safe evaluation configuration reduced information leakage during testing and encouraged structural RTL reasoning during Trojan detection and localization analysis.

V. RESULTS AND DISCUSSION

A. Detection Performance

To test the performance of the proposed framework, a leakage-safe randomized testing methodology was used to evaluate unseen RTL samples from the stratified test dataset. During evaluation, inference was performed using prompt-only RTL inputs while classification target labels remained hidden from the adapted model. The evaluation process included balanced distributions of clean and Trojan-inserted RTL samples in order to reduce classification bias during binary RTL Trojan detection.

The obtained results demonstrated that the proposed framework could consistently detect RTL Trojans across the randomized leakage-safe test set while maintaining high accuracy, precision, recall, and F1-score performance. The precision results indicated highly reliable Trojan predictions during inference, while the recall values showed that the majority of Trojan-inserted RTL samples were successfully detected within the controlled experimental setup.

Confusion matrix analysis indicated that all clean RTL samples were correctly classified without false-positive predictions, while only one Trojan-inserted RTL sample was misclassified as clean. These observations demonstrated balanced and consistent binary classification behavior under the leakage-safe evaluation workflow.

B. Localization Performance

A localization-aware evaluation was performed in which the adapted model attempted to identify RTL regions exhibiting structural suspiciousness associated with Trojan trigger activation and payload behavior by comparing predicted RTL trigger and payload regions against tracked ground-truth metadata generated during synthetic Trojan insertion. Trigger Localization Coverage (TLC) and Payload Localization Coverage (PLC) metrics similar to those used in recent LLM-based RTL Trojan analysis studies were used to evaluate localization performance [4], [5].

TABLE V
LOCALIZATION PERFORMANCE METRICS

Metric	Value
Trigger Localization Coverage (TLC)	98.89%
Payload Localization Coverage (PLC)	98.89%

The localization evaluation results indicated a high degree of overlap consistency between predicted RTL regions and the corresponding ground-truth trigger and payload metadata. Both TLC and PLC metrics produced balanced localization behavior under the randomized leakage-safe evaluation workflow. The overlap-based localization methodology also demonstrated the effectiveness of metadata-guided localization analysis within the controlled synthetic RTL Trojan environment used throughout the experiments.

The localization results demonstrated that the proposed framework could support localization-aware RTL security analysis beyond binary Trojan classification alone. Although the evaluation was performed within a controlled synthetic RTL Trojan setting, the overlap-based localization workflow provided practical insight into structurally suspicious RTL regions associated with Trojan trigger and payload behavior.

C. Resource Efficiency Analysis

The lightweight parameter-efficient adaptation framework proposed for low-resource RTL Trojan analysis features a workflow based on LoRA-based PEFT adaptation integrated with the Qwen2.5-1.5B-Instruct model, resulting in reduced numbers of trainable parameters compared to conventional RTL adaptation capabilities. The adapted framework can execute on Google Colab using NVIDIA Tesla T4 GPUs and produce reproducible results without requiring a dedicated high-end computing infrastructure.

The reduction in trainable parameter count from the lightweight adaptation resulted in reduced GPU memory allocation and computation overhead during training. The entire training workflow was executed in a short amount of time and is feasible for consumer-accessible academic environments, improving reproducibility and practical accessibility for smaller research laboratories and engineering-oriented experimentation with respect to RTL security analysis.

Unlike PEARL, which investigated methods for enterprise and open-source LLM adaptations for RTL Trojan analysis, the core focus of the proposed framework is on lightweight parameter-efficient adaptation under constrained hardware environments [4], [6], [13]. Rather than directly compete with PEARL, the proposed workflow represents a complementary research direction emphasizing low-resource reproducibility and practical PEFT-style experimentation for RTL Trojan detection tasks. PEARL is therefore treated as a contextual

TABLE VI
RESOURCE EFFICIENCY COMPARISON

Aspect	PEARL	Proposed
Adaptation	Full/ICL LLM	LoRA-based PEFT
GPU	RTX 3090 Ti	Tesla T4
Platform	Workstation	Google Colab
Trainable Params	Full/High	2.17M
Trainable %	N/R	0.141%
Runtime	Higher	$\sim$6 min
Focus	High-performance	Resource-efficient

reference rather than a directly comparable benchmark because its model, adaptation strategy, and evaluation setting differ from the controlled LoRA-based setup used here.

D. Limitations

A controlled synthetic RTL Trojan dataset was used to evaluate the proposed framework using lightweight trigger and payload insertion techniques. The experimental design allowed reproducible evaluation of RTL Trojan analysis and localization; however, the current study remains limited to RTL-level synthetic Trojan representations within a controlled low-resource environment. Similar controlled RTL evaluation methodologies have also been explored in recent LLM-based hardware security studies [4].

The current implementation was not used to evaluate post-synthesis gate-level Trojan analysis, physical design validation, or large-scale industrial deployment scenarios. In addition, the framework was evaluated primarily using structurally generated RTL Trojan samples rather than heterogeneous real-world hardware security benchmarks. The randomized stratified split preserves class balance but does not explicitly enforce template-level separation; therefore, structural similarity across training and testing samples cannot be ruled out. Therefore, the proposed workflow should be interpreted as a lightweight engineering-oriented prototype for reproducible RTL Trojan experimentation rather than a production-scale hardware security framework.

Future extensions may include evaluation across larger benchmark collections, incorporation of real-world Trojan datasets, and broader adaptation studies involving additional hardware security analysis tasks and transformer-based RTL reasoning workflows.

VI. CONCLUSION

This research introduced a lightweight framework for RTL Hardware Trojan analysis using parameter-efficient LoRA adaptation of the Qwen2.5-1.5B-Instruct model for RTL security assessment. The proposed workflow combined synthetic Trojan generation, obfuscation-aware preprocessing, instruction-style prompt preparation, and localization-aware evaluation within a low-resource experimental environment. In addition to binary Trojan classification, the framework also supported overlap-based trigger and payload localization analysis using TLC and PLC methodologies.

Experimental results demonstrated reliable RTL Trojan detection and effective localization capability under the leakage-safe randomized evaluation workflow. The lightweight LoRA adaptation strategy enabled reproducible experimentation using a Google Colab NVIDIA Tesla T4 GPU while maintaining low computational overhead and significantly reduced trainable parameter requirements compared with conventional large-scale adaptation workflows [4], [6], [13].

Overall, the proposed framework demonstrated that lightweight parameter-efficient adaptation can support reproducible RTL hardware security analysis and academic experimentation while also providing a practical foundation for broader transformer-based hardware security research under constrained computational environments.

REFERENCES

[1] M. Tehranipoor and F. Koushanfar, "A survey of hardware trojan taxonomy and detection," *IEEE Design & Test of Computers*, vol. 27, no. 1, pp. 10–25, 2010.

[2] Y. Jin and Y. Makris, "Hardware trojan detection using path delay fingerprint," in *IEEE International Workshop on Hardware-Oriented Security and Trust (HOST)*, 2008, pp. 51–57.

[3] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[4] R. K. Kundu, K. Khalil, E. Garcia, E. Grassia, P. Calyam, and K. A. Hoque, "Pearl: An adaptive and explainable hardware trojan detection using open-source and enterprise large language models," *IEEE Access*, vol. 13, pp. 133 755–133 772, 2025.

[5] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Trojanwhisper: Evaluating pre-trained llms to detect and localize hardware trojans," *IEEE Access*, vol. 14, pp. 30 965–30 983, 2026.

[6] E. J. Hu *et al.*, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[7] V. T. Hayashi and W. V. Ruggiero, "Hardware trojan detection in open-source hardware designs using machine learning," *IEEE Access*, vol. 13, pp. 53 200–53 217, 2025.

[8] G.-W. Wan, S.-Z. Wong, D. Liu, and X. Wang, "Assessment of pre-trained large language models for hardware trojan detection in rtl designs," in *IEEE LLM Aided Design Workshop (LAD)*, 2024, pp. 1–6.

[9] S. Liu *et al.*, "Rtl-coder: A new llm solution for rtl code generation," https://github.com/hkust-zhiyao/RTL-Coder, 2024.

[10] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of EMNLP*, 2020, pp. 38–45.

[11] H. Face, "Peft: Parameter-efficient fine-tuning of large language models," https://github.com/huggingface/peft, 2023.

[12] T. Dettmers, "bitsandbytes," https://github.com/TimDettmers/bitsandbytes, 2022.

[13] T. Dettmers *et al.*, "Qlora: Efficient finetuning of quantized llms," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[14] A. Paszke *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *NeurIPS*, 2019.